Odin OS: Independent Verification & Falsification Guide

Claim-to-Evidence Mapping, Trust Assumptions, and Reproducibility Protocols

Nathan Walker
Ravenhelm Platform & Runestack Research
Email: nwalker85@gmail.com | ORCID: 0009-0005-0965-4424
Permanent Archive DOI: 10.5281/zenodo.23928302 | Repository: https://github.com/nwalker85/odinos
Canonical Phase B.3.2 Baseline — Revision 68f45c1 — November 7, 2026

Peer-Review Mandate & Epistemological Scope Boundary

This document provides a structured basis for independent technical review and falsification. It does not constitute independent certification, a completed external audit, or proof of semantic correctness.

The objective of this assurance package is to make every significant technical claim reported in the Odin OS research literature independently inspectable, reproducible, and falsifiable. A reviewer should be able to identify a specific claim, examine its trusted assumptions, locate the supporting source implementation, execute the relevant experiments, and attempt to disprove the claim without relying on author assertions.

1 Purpose & Review Scope

Operating system security research frequently reports empirical success matrices and passing test suites, yet reviewers are rarely equipped with bounded starting points, exact failure conditions, or the architectural boundaries needed to systematically test the limits of those claims.

This assurance guide accompanies the Odin OS Phase B.3.2 publication release candidate. It directly addresses the core challenge received during initial peer consultation:

> *"For the peer review you are seeking, a claim-to-evidence map would help a reader choose a bounded starting point: the exact property, assumptions and trusted components, then the artifact or test supporting it and what remains outside scope. Which single claim would you most like an independent reviewer to try to falsify first? In particular, where do you draw the boundary between enforcing an authorized memory access and establishing that the resulting business action is appropriate? Those seem like distinct review questions."*

This guide resolves that challenge by:

1. **Disentangling Access Enforcement from Business Policy:** Clarifying the formal boundary between the Policy Enforcement Point (microkernel PEP) and Policy Decision Point (PDP).
2. **Establishing a Priority Falsification Target:** Providing reviewers with a single, razor-sharp architectural invariant to challenge (`ODIN-C001`).
3. **Ensuring End-to-End Traceability:** Mapping 14 granular claims to source code line ranges, test suites, formal SMT models, and SHA-256 evidence digests.

2 Evidence Status Definitions

In accordance with scientific rigor, every technical claim in Odin OS is assigned an explicit, conservative assurance status:

- **`REPRODUCED`**: Directly executed and verified during clean-room evaluation on this specific commit (`68f45c1`).
- **`MODEL-CHECKED`**: Formally proved in Z3 SMT (`QF_BV`). Proves mathematical bitvector abstractions, not compiled machine binaries.
- **`QUALIFIED`**: Valid under explicit, documented operating assumptions (e.g., software emulation under QEMU 11.1.3 `virt`); known hardware differences on physical silicon require additional mitigations.
- **`PROPOSED`**: Architectural specification for future roadmap phases; not implemented in the current prototype.
- **`REPORTED`**: Documented in project history without fresh independent re-execution.
- **`CONTRADICTED`**: Evidence conflicts with or refutes the claim.

3 System Trust Boundaries & The 4 Assurance Layers

Odin OS explicitly separates system guarantees across four operational layers. A guarantee at one layer does not establish security at another.

Layer D: Semantic & Business Appropriateness (Application Domain)
Question: "Was the resulting action appropriate under business rules?"
Authority: Upstream Application / Enterprise Governance / Human Supervisor.
Guarantee: NONE from Microkernel. Hardware PMP does not evaluate business intent.

Layer A: Cryptographic Authorization (Policy Decision Point - PDP)
Question: "Did the trusted authority sign this specific mutation request?"
Authority: Policy Decision Point (`pdp_authority_service.py`), Root Ed25519 Private Key.
Guarantee: Capability minting requires declarative policy satisfaction; monotonic anti-replay sequences.

Layer B: Hardware Enforcement (Policy Enforcement Point - PEP)
Question: "Can an untrusted instruction commit stores outside active bounds?"
Authority: RISC-V PMP Comparators & M-Mode Trap Handler (`src/main.rs`).
Guarantee: Architectural Store Access Fault (`mcause = 7`) aborts bus commit before memory mutation.

Layer C: Transactional & Content Integrity (Memory Controller Barrier)
Question: "Does physical memory match the cryptographic post-state digest?"
Authority: Dual-buffer Shadow AST & SHA-256 Commit Barriers (Phase C Specification).
Guarantee: Prototype enforces spatial byte windows, not cryptographic digest post-state equality.

Listing 1: The Four Assurance Layers of Odin OS.

3.1 Component Trust Classifications

- **CPU Hardware (PMP Comparators)**: **Trusted Enforcer**. Enforces address comparator matches, raises access faults, and enforces privilege modes.
- **M-Mode Microkernel (TCB)**: **Trusted Verifier & Enforcer**. Verifies Ed25519 signatures against pinned public key, programs PMP registers, and handles traps.
- **Policy Decision Point (PDP)**: **Trusted Authority**. Evaluates agent mutation requests against declarative security policies, manages sequence counters, and holds root private key.
- **Layer B External Verifier**: **Auditing Verifier**. Independently audits receipt schemas and trace continuity.
- **Autonomous Agent Task (U-Mode)**: **Untrusted Adversary**. Executes model tokens and arbitrary arithmetic. Zero private keys; zero capability to reprogram CSRs.
- **Host Transport Bridge**: **Untrusted Transport Channel**. Transports model tokens and capability requests; assumed fully compromiseable.

4 Priority Review Target: ODIN-C001 (Spatial Mutation Isolation)

An independent reviewer seeking a bounded starting point should prioritize:

ODIN-C001: Spatial Mutation Isolation Invariant

Narrow Invariant Statement: Under the tested QEMU 11.1.3 `virt` configuration (`riscv64gc-unknown-none-elf`), an unprivileged User Mode (U-Mode) task executing arbitrary instructions cannot commit a memory store to any physical address outside the explicitly configured Top-of-Range (TOR) window `[lease_start, lease_end)` without triggering an architectural Store Access Fault (`mcause = 0x07`) that aborts the bus write and leaves the probed memory byte unmodified.

4.1 The Six Empirical Review Questions for ODIN-C001

To rigorously falsify ODIN-C001, an independent reviewer should examine these six concrete questions:

1. **Is a cryptographically verified deed required before a writable lease is established? Finding:** Yes. Microkernel boot configures memory with Default Deny (`---`). The only mechanism to establish a writable window is submitting a valid TransitionDeed via `sys_verify_deed()` or hardware loader.
2. **Do the deed's authorized bounds match the PMP hardware configuration? Finding:** Yes. PMP Entry 4 is programmed with `target_offset` and `target_offset + target_length` mapped to physical DRAM base `0x80020000`.
3. **Do unauthorized stores fault? Finding:** Yes. Empirical tests 3, 4, 5, 6, 7, 9, 10, 16, and 17 all trigger hardware Store Access Faults (`mcause = 0x07`).
4. **Do bytes remain unmodified after faulting stores? Finding:** Yes. Probed memory readbacks confirm target addresses retain original values (`0x00` or original executable code).
5. **Can User Mode manipulate PMP registers? Finding:** No. Unprivileged execution of `csrr`/`csrw pmpcfg0` traps with Illegal Instruction (`mcause = 0x02`).
6. **Does revocation remove write permissions? Finding:** Yes. Calling syscall 13 (`SYS_REVOKE_LEASE`) resets PMP Entry 4 to `OFF`; subsequent writes immediately fault.

4.2 Scientific Boundaries & Non-Claims for ODIN-C001

- **What ODIN-C001 Establishes:**
  - An untrusted agent cannot write outside its active deed slice under QEMU emulation.
  - Privilege escalation via stack execution or CSR manipulation is prevented by hardware.
- **What ODIN-C001 Does NOT Establish:**
  - It does not prove that the bytes written inside the lease are semantically correct or safe for the application.
  - It does not prove that physical silicon cores will exhibit atomic split-store behavior across protection perimeters without shadow AST staging buffers (see Limitation 1).
  - It does not prove protection against microarchitectural cache-timing side channels (Spectre/Meltdown).

5 Granular Claim-to-Evidence Matrix

The table below catalogs all 14 technical claims across the 4 Assurance Layers:

ID	Exact Claim	Layer	Status	Implementation & TCB	Supporting Artifact / Test	Protocol
C001	Spatial Mutation Isolation	PEP (B)	**REPRODUCED**	PMP Entry 4 (TOR); src/main.rs:252	Matrix Tests 1–4, 9, 10; qemu-adversarial-matrix.txt	PROTO-01
C002	Decoupled PDP Authority	Auth (A)	**REPRODUCED**	Host Python PDP; tools/pdp_authority_service.py	12 unit tests in authority-isolation-suite.txt	PROTO-06
C003	Pure #[no_std] Ed25519	Auth (A)	**REPRODUCED**	ed25519-compact; src/deed.rs:185	Matrix Tests 11, 13, 14; 23 host unit tests	PROTO-06
C004	Anti-Replay Nonce	Auth (A)	**MODEL-CHECKED**	Kernel sequence counter; Z3 sequence_anti_replay	Matrix Test 12; Z3 Theorem 4 & 5 (smt-proofs.txt)	PROTO-07
C005	Bi-directional $W \oplus X$	PEP (B)	**REPRODUCED**	PMP Entry 1 (R-X), Entry 2 (RW-); trap handler	Matrix Tests 7 & 8; Mutations M06 & M15	PROTO-04
C006	Temporal Revocation	PEP (B)	**REPRODUCED**	Syscall 13 (SYS_REVOKE_LEASE); src/main.rs:1015	Matrix Test 16; Mutation M09	PROTO-03
C007	CSR & MMIO Isolation	PEP (B)	**REPRODUCED**	PMP Entry 0 (MMIO ---); M-Mode CSR gating	Matrix Tests 15 & 17; Mutation M07	PROTO-05
C008	Multi-Byte Crossing	PEP (B)	**QUALIFIED**	QEMU softmmu split access check	Matrix Test 10 (0x8002005E, all 4B unmodified)	PROTO-02
C009	Bounded Arithmetic	Integ (C)	**MODEL-CHECKED**	Host checked_add; Z3 deed_bounds_verification; prop_deed_overflow	Z3 Theorem 1 (unsat);	PROTO-08
C010	Layer B External Trace	Integ (C)	**REPRODUCED**	Python verifier; tools/verify_evidence.py	10 adversarial tests; layer-b-evidence-verifier.txt	Audit Run
C011	Shadow AST Commit	Integ (C)	**PROPOSED**	Dual-buffer commit barrier specification	Design doc only (PHASE_B3_SEMANTIC_INTEGRITY)	Phase C
C012	Spatial != Semantic	Seman (D)	**INVARIANT**	Decoupled PDP/PEP architecture	Axiomatic: Agent Request != Authority Grant	PROTO-09
C013	Prompt Injection Defense	Seman (D)	**REPRODUCED**	Gemma 4 bridge; tools/gemma_odin_bridge.py	Bridge Scenarios 1 & 2 (gemma-bridge-execution.txt)	Bridge Run
C014	Physical Hardware	PEP (B)	**PROPOSED**	Kria KV260 & JH7110 roadmap	Design doc only (HARDWARE_MODIFICATION_KRIA)	Phase C

Table 1: Granular Claim-to-Evidence Matrix for Odin OS Phase B.3.2 Baseline.

6 Structured Assurance Argument (GSN-Informed)

To prevent circularity, the assurance argument decomposes the primary goal into independent sub-goals supported by distinct evidence layers:

Goal G1: Spatial & Temporal Memory Isolation for Untrusted Agent Execution
Context C1: RISC-V 64-bit PMP Architecture | **Assumption A1:** QEMU 11.1.3 PMP softmmu matches Privileged ISA v1.12

Strategy S1: Defense-in-Depth Decomposition (Authorization + Hardware PEP + Formal Models)

- **Sub-Goal G1.1 (Cryptographic Authorization):** No lease programmed without valid Ed25519 root signature.
  - **Evidence E1.1:** In-kernel `ed25519-compact` verifier; Matrix Tests 11, 13, 14; 23 host unit tests.
- **Sub-Goal G1.2 (Hardware Spatial Confinement):** Out-of-bounds stores abort before memory commit.
  - **Evidence E1.2:** Processor PMP Entry 4; Matrix Tests 1–4, 9, 10; zero probed bits modified.
- **Sub-Goal G1.3 (Temporal Lifecycle):** Capability expiration immediately revokes write permissions.
  - **Evidence E1.3:** Microkernel syscall 13; Matrix Test 16; Mutation M09.
- **Sub-Goal G1.4 (Privilege Separation):** Code is immutable ($W \oplus X$); stack non-executable; CSRs isolated.
  - **Evidence E1.4:** PMP Entries 1, 2, 0; Matrix Tests 7, 8, 15, 17; safe stack trap redirection.
  - **Sub-Goal G1.5 (Arithmetic Safety):** Integer offsets cannot overflow buffer bounds.
  - **Evidence E1.5:** Rust `checked_add`; Z3 Theorem 1 (`unsat` in 0.02s); proptest bounds check.

Unresolved Proof Obligations:
(1) Split-store atomicity across protection boundaries on physical silicon cores (governed by RISC-V ISA §2.6).
(2) Autonomous preemption via hardware timer interrupts (`mcycle`/`mtimecmp`) without requiring explicit syscalls.
(3) Content post-state digest verification via in-kernel SHA-256 and dual-buffer shadow AST staging.

Listing 2: Goal Structuring Notation (GSN) Informed Assurance Tree for Odin OS.

7 Clean-Room Fast-Path & Step-by-Step Reproduction

All reported results can be reproduced on a standard workstation in under 5 seconds:

```
# 1. Execute Canonical Release Verification Pipeline (All 9 Gates Unattended)
python3 tools/verify_release.py

# 2. Run the 17-Case QEMU Bare-Metal Adversarial Matrix
cargo run --release

# 3. Verify Formal SMT Proofs in Z3 (QF_BV)
z3 formal/deed_bounds_verification.smt2 && z3 formal/sequence_anti_replay.smt2

# 4. Run Host Unit Tests & Property Fuzzing (23 tests)
cargo test --lib --target aarch64-apple-darwin

# 5. Run Independent Layer B Trace Auditor
python3 tools/verify_evidence.py
```

8 Actionable Reviewer Falsification Protocols

The following protocols provide explicit recipes for independent researchers attempting to disprove stated invariants:

8.1 PROTO-01: Spatial Boundary Breach (ODIN-C001)

- **Objective:** Falsify the claim that U-Mode cannot commit memory writes outside active deed bounds.
- **Procedure:** In `src/main.rs`, modify `user_mode_adversarial_suite()` to issue a store to `0x8002003F` (1 byte below lease) or `0x80020060` (1 byte above lease). Run `cargo run --release`.
- **Expected Behavior:** CPU raises Store Access Fault (`mcause = 7`); readback confirms byte is `0x00`.
- **Falsification Condition:** Falsified if target memory updates to written byte, or store executes without `mcause = 7`.

8.2 PROTO-02: Multi-Byte Overlap Partial Commit (ODIN-C008)

- **Objective:** Falsify the claim that a multi-byte store crossing a protection boundary does not fracture.
- **Procedure:** In U-Mode, execute `sw 0xDEADBEEF, 0(0x8002005E)` crossing into unmapped perimeter `0x80020060`. Inspect bytes across `0x8002005E..0x80020062`.
- **Expected Behavior:** Under QEMU virt, access faults atomically with `mcause = 7`; all 4 bytes remain `0x00`.
- **Falsification Condition:** Falsified if lower bytes commit while upper bytes trap (partial commit / atomic fracture).

8.3 PROTO-03: Temporal Capability Leakage (ODIN-C006)

- **Objective:** Falsify the claim that lease revocation immediately terminates write capability.
- **Procedure:** Perform valid store to `0x80020040`; invoke `sys_revoke_lease()`; immediately attempt store `sb 0x99, 0(0x80020040)`.
- **Expected Behavior:** Second store triggers `mcause = 7`; target byte remains `0x41`.
- **Falsification Condition:** Falsified if post-revocation store succeeds without fault.

8.4 PROTO-04: Bi-Directional $W \oplus X$ Privilege Bypass (ODIN-C005)

- **Objective:** Falsify the claim that user code is immutable and user stack is non-executable.
- **Procedure:** Attempt store to user code `0x80010000`; attempt to jump to instruction `0x00008067` written to stack `0x80018100`.
- **Expected Behavior:** Code store faults with `mcause = 7`; stack execute faults with `mcause = 1` and redirects safely.
- **Falsification Condition:** Falsified if code byte mutates, or code executes from user stack without fault.

8.5 PROTO-05: PMP CSR Tampering from User Mode (ODIN-C007)

- **Objective:** Falsify the claim that U-Mode cannot modify or inspect PMP configuration.
- **Procedure:** In U-Mode, execute `csrr t0, pmpcfg0` or `csrw pmpcfg0, zero`.
- **Expected Behavior:** Processor raises Illegal Instruction trap (`mcause = 2`).
- **Falsification Condition:** Falsified if `csrr` reads configuration or `csrw` clears protection windows.

8.6 PROTO-06: Cryptographic Signature Forgery (ODIN-C003)

- **Objective:** Falsify the claim that forged or single-bit-flipped TransitionDeeds are rejected.
- **Procedure:** Invert a single bit in the 64-byte signature block of a valid deed and call `sys_verify_deed()`.
- **Expected Behavior:** Microkernel verifier returns `DeedError::InvalidSignature` (error code 4).
- **Falsification Condition:** Falsified if `sys_verify_deed()` returns `0` (Success).

8.7 PROTO-07: Monotonic Anti-Replay Sequence Nonce (ODIN-C004)

- **Objective:** Falsify the claim that replayed TransitionDeeds cannot reopen memory windows.
- **Procedure:** Submit deed with `sequence = 100`; immediately resubmit identical deed with `sequence = 100`.
- **Expected Behavior:** Microkernel verifier returns `DeedError::ReplayDetected` (error code 2).
- **Falsification Condition:** Falsified if resubmission returns `0` or programs PMP registers.

8.8 PROTO-08: Integer Overflow Lease Clamping (ODIN-C009)

- **Objective:** Falsify the claim that integer wrap attacks on lease bounds are blocked.
- **Procedure:** Construct deed with `target_offset = 0xFFFFFFF0` and `target_length = 0x20`.
- **Expected Behavior:** Rust `checked_add` fails closed (`DeedError::OutOfBoundsLease`); Z3 model returns `unsat`.
- **Falsification Condition:** Falsified if lease wraps around to low memory, or Z3 outputs `sat`.

8.9 PROTO-09: Semantic In-Bounds Mutation (ODIN-C012)

- **Objective:** Demonstrate the fundamental boundary between spatial capability and business correctness.
- **Procedure:** Obtain valid deed for `[0x40..0x60]`. Write arbitrary corrupt bytes inside this 32-byte window.
- **Observation:** Store completes without fault (`mcause = 0`). Confirms that hardware enforcement does not guarantee semantic correctness.

8.10 PROTO-10: Unauthorized UART0 MMIO Access (ODIN-C007)

- **Objective:** Falsify the claim that untrusted U-Mode code cannot hijack terminal console output via MMIO.
- **Procedure:** In U-Mode, attempt direct store: `sb 0x58, 0(0x10000000)` (UART0 transmitter holding register).
- **Expected Behavior:** Processor triggers Store Access Fault (`mcause = 7`); console emits no rogue characters.
- **Falsification Condition:** Falsified if character `X` (`0x58`) is output to terminal without fault.

8.11 PROTO-11: Compromised Transport Channel Injection (ODIN-C002)

- **Objective:** Falsify the claim that transport channels cannot forge capability grants without root keys.
- **Procedure:** Inject an out-of-bounds mutation token directly into the Python bridge (`gemma_odin_bridge.py`) targeting `0x80030000`.
- **Expected Behavior:** PDP policy rejects deed issuance; unauthenticated store in U-Mode is intercepted by PMP.
- **Falsification Condition:** Falsified if bridge mints a valid signature or causes kernel to commit write.

8.12 PROTO-12: Sequence Nonce Reset on Cold Reboot (ODIN-C004 Boundary)

- **Objective:** Demonstrate the persistence boundary limitation of the in-memory sequence counter.
- **Procedure:** Consume deed sequence `100`; execute hypervisor reset (`qemu-system-riscv64` reboot); resubmit deed sequence `100`.
- **Observation:** Sequence is accepted because RAM counter was reset to `0`. Confirms documented Limitation 4.

9 Known Limitations & Unresolved Proof Obligations

1. **PMP Grain Size ($G > 0$ on Physical Silicon):** QEMU virt implements $G = 0$ (byte granularity). Physical silicon with $Ge4$ enforces 16-byte alignment bounds.
2. **Write-Buffer Draining on Pipelined Cores:** Pipelined physical silicon requires explicit `fence rw, rw` barriers during PMP reprogramming.
3. **Misaligned Access Atomicity:** RISC-V ISA §2.6 permits non-atomic split transfers; physical deployment mandates shadow AST staging buffers.
4. **Autonomous Preemption:** Capability leases currently require explicit syscall revocation; hardware timer interrupts (`mcycle`/`mtimecmp`) are designed for Phase C.
5. **Spatial vs. Content Integrity:** Current prototype enforces spatial write windows; in-situ SHA-256 state hashing is proposed architecture.
6. **Non-Volatile Sequence Nonce Persistence:** Sequence counter in RAM resets on cold reboot without non-volatile NVRAM.
7. **Formal Model Abstraction Gap:** Z3 SMT proves mathematical bitvector specifications, not compiled binary instructions.
8. **Root Key Custody:** Mode B relies on host OS process and filesystem security (`chmod 600`) without hardware HSM/TPM enclaves.

10 Reviewer Findings & Responsible Disclosure

If you identify an empirical counterexample, an unhandled hardware trap, or an invariant violation:

1. Copy the template in [`peer-review/REVIEWER_FINDINGS_TEMPLATE.md`](https://github.com/nwalker85/odinos/blob/main/peer-review/REVIEWER_FINDINGS_TEMPLATE.md).
2. Record the Git commit SHA, toolchain versions, minimal reproducer patch, and register dump (`mepc`, `mcause`, `mtval`, probed address, readback value).
3. Submit a disclosure report via GitHub Issues: `https://github.com/nwalker85/odinos/issues` or email `nwalker85@gmail.com`.

11 Artifact Provenance & Traceability

All evidence artifacts are recorded in `peer-review/evidence-manifest.json` with cryptographic SHA-256 digests:

Artifact ID	Relative Path	SHA-256 Digest	Status
EV-001	peer-review/evidence/qemu-adversarial-matrix.txt	4b08be3f15d1781ccbc44aa63c287782b…	VERIFIED
EV-002	peer-review/evidence/host-unit-tests.txt	9d4ffd60ff4afa2338b5bc107646eaec8…	VERIFIED
EV-003	peer-review/evidence/smt-proofs.txt	61cbbfc39ef24209ea4eb48ef450c437…	VERIFIED
EV-004	peer-review/evidence/authority-isolation-suite.txt	f29f7827b66a1937eec363ec247bf347…	VERIFIED
EV-005	peer-review/evidence/verifier-adversarial-suite.txt	38eeff1237d74275801617a271eced659…	VERIFIED
EV-006	peer-review/evidence/layer-b-evidence-verifier.txt	02c047de550593357b2124ebcc6bf08a…	VERIFIED
EV-007	peer-review/evidence/release-verification-pipeline.txt	87cabd7de80a36e33fc6f77e5aa355d48…	VERIFIED
EV-008	peer-review/evidence/gemma-bridge-execution.txt	2ab695abe4b118818364569b17c3a306…	VERIFIED
EV-009	zenodo/receipts/adversarial_matrix_receipt.json	a3fabc7fcaf6770587…	AUTHENTICATED
EV-010	zenodo/receipts/raw_execution_trace.txt	fef3815426fa6430356e3154633b22ec…	AUTHENTICATED

Table 2: Cryptographic Evidence Manifest and SHA-256 Provenance Ledger.

—